Macro expansion: hide-set bounds, levels, and primitive recursion

1 Reduction rules

Tokens are a^H: spelling a, hidden names H. Sequences U, V are joined by juxtaposition; ϵ is empty. Fresh tokens have $H = \emptyset$. The environment contains $\Gamma(m) = B$ or $\Gamma(m) = (\vec{x}, B)$; all operations use this same Γ.

$$M_K(a^H) = a^{H \cup K}, \qquad F(a^H) = \begin{cases} a^{\{a\}} & a \in H, \\ a^{\emptyset} & a \notin H \end{cases} \qquad (M, F \text{ act tokenwise}).$$

Scan. A head m^H is *eligible* when $m \notin H$, $\Gamma(m)$ is defined, and a function-like macro is immediately followed by `(`.

$$E(U) \to F(S(U)), \qquad S(\epsilon) \to \epsilon, \qquad S(a^H U) \to a^H\, S(U) \ \text{ if the head is not eligible,}$$

$$\frac{\Gamma(m) = B \quad m \notin H}{S(m^H U) \to S\big(M_{\{m\} \cup H}(P(B))\, U\big)} \ \text{(object)} \qquad \frac{\Gamma(m) = (\vec{x}, B) \quad m \notin H \quad \vec{A} \text{ binds } \vec{x}}{S\big(m^H(\vec{A})^J U\big) \to S\big(M_{\{m\} \cup (H \cap J)}(P(\mathrm{sub}_{\vec{A}}(B)))\, U\big)} \ \text{(function)}$$

Arguments split at commas outside nested parentheses; the variadic slot is the comma-joined suffix. J belongs to the matching closing parenthesis.

Substitution. For each original parameter occurrence x_i in B: $\mathrm{sub}_{\vec{A}}(x_i) = A_i$ if adjacent to `##` and $A_i \neq \epsilon$; $\square$ if adjacent to `##` and $A_i = \epsilon$; $E(A_i)$ otherwise. Other body tokens are copied. Each required $E(A_i)$ is computed on A_i alone, before rescanning; unused arguments are not expanded.

Pasting. P reduces the original `##` operators of the replacement list left to right, then erases placemarkers: $a^H \,\texttt{\#\#}\, b^J \to (ab)^{H \cap J}$, $\square \,\texttt{\#\#}\, a^H \to a^H$, $a^H \,\texttt{\#\#}\, \square \to a^H$, $\square \,\texttt{\#\#}\, \square \to \square$; ab is a single token, defined only when it exists. For multi-token operands only the adjacent boundary tokens are pasted; generated tokens never become paste operators.

Constants of Γ. $\mathcal{M} = \mathrm{dom}\,\Gamma$, $n_\Gamma = |\mathcal{M}|$; L is the maximal body length; k the maximal number of occurrences of one parameter in a body.

2 Origins and the trick

Definition 1. Every rewrite step e by the object or function rule has the *mark* $P_e = \{m\} \cup H$, resp. $\{m\} \cup (H \cap J)$. Every token of $M_{P_e}(\cdots)$ produced by e, including substituted argument tokens, has *origin* $\mathrm{orig} = e$. Input tokens have origin $\perp$, $P_\perp = \emptyset$. Origins are ordered by production: $e \prec e'$ if e' is performed while scanning a token sequence containing the head of e' produced (transitively) inside e.

Definition 2. An invocation is a *trick step* if its closing parenthesis has an origin strictly outer than the origin of its head. A run without trick steps is *trick-free*. The *trick* of the introduction (macros returning one another, fed with parenthesised groups) is exactly a sequence of trick steps; the groups may come from the text after the call or from arguments copied into a replacement — both are of outer origin.

Lemma 3. *A token of origin e has hidden set $\supseteq P_e$ while it is scanned.*

Proof. M_{P_e} is applied after P and after substitution of the already-finalised $E(A_i)$, hence to every token of the replacement; scanning does not rewrite hidden sets of tokens it passes. $\square$

Lemma 4. *If an invocation of m is not a trick step and its head has origin e, then $m \notin P_e$ and the new mark is $\supseteq P_e \cup \{m\}$.*

Proof. By Lemma 3, $H \supseteq P_e$ and $J \supseteq P_e$, so $H \cap J \supseteq P_e$; eligibility gives $m \notin H$. $\square$

Lemma 5. *In a trick-free invocation whose head has origin e, every argument token has origin e.*

Proof. Scanning is left to right; at the head, the stream is the unscanned rest of the replacement of e followed by outer material. The closing parenthesis has origin e, so it and everything between lie in the first part. $\square$

Proposition 6 (trick-free bound)**.** *If the input is one invocation whose arguments contain no name of $\mathcal{M}$ and the run is trick-free, the number of rewrite steps is at most a constant $C(n_\Gamma, L, k)$, and the output length is at most $a\,|\mathit{input}| + b$ with a, b depending on Γ only.*

Proof. Let the parent of a step be the origin of its head. By Lemma 4 marks strictly increase along parent chains, so the step tree has depth $\le n_\Gamma$. Let N_t bound the number of names of $\mathcal{M}$ among tokens of origin e for e at depth t; children of e are heads of origin e, so branching at depth t is $\le N_t$. Input arguments carry no names, hence $N_0 \le L$. For depth $t \ge 1$, by Lemma 5 arguments come from the parent's tokens ($\le N_{t-1}$ names) and from replacements of siblings expanded inside them during $E(A_i)$; each sibling uses a strictly smaller argument region of the same A_i, so a recursion on the number $j \le N_{t-1}$ of enclosed heads, $Q_t(j) \le L + Lk\,(N_{t-1} + \max_{\sum j_i < j} \sum_i Q_t(j_i))$, is finite and gives $N_t \le Q_t(N_{t-1})$. Hence $\#\text{steps} \le \sum_{t \le n_\Gamma} \prod_{s<t} N_s =: C$. Name-free tokens are only copied ($\le k$ times per parameter per step), which yields the linear bound on length. □

Corollary 7. *Along a parent chain, marks lose names only through $H \cap J$ with J of outer origin, i.e. only at trick steps (Lemma 4). Trick steps are therefore the only source of dependence of the number of steps on the input.*

Remark 8. Hidden sets of individual tokens lose names in a second way: F keeps only a token's own spelling, so every token of a pre-expanded argument leaves $E(A_i)$ with $H \subseteq \{a\}$ (a `)` with $H = \emptyset$). Lemma 3 is not affected, since M_{P_e} is applied after F; such a loss shows only later, as a small J in some $H \cap J$, that is, at a trick step.

3 Levels and the upper bound

Definition 9. The call graph G_Γ has an edge $m \to m'$ when m' is spelled in a token of the body of m (directly, or as a possible paste result). A *pass set* is a strongly connected component of G_Γ; it is *non-trivial* when it contains a cycle (two or more macros, or one with an edge to itself). The level $\mathrm{lev}(m)$ is the largest number of non-trivial components on a path of the condensation DAG starting at the component of m, that component included. So lev $= 0$ exactly when no cycle is reachable; the pass `GEN_MULTIPLY_2_A/B` has level 1, and `TGEN_EXP_A/B`, which reaches it, level 2. Levels are bounded by n_Γ and fixed by the source: expansion never creates definitions, since a macro-replaced sequence is not processed as a directive (C11 6.10.3.4p3).

Lemma 10 (cycles need the trick)**.** *Along a parent chain without trick steps every macro occurs at most once. Consequently a macro of a pass set can be re-invoked below its own active invocation only through a trick step.*

Proof. By Lemma 4 each step of the chain adds its macro to the mark and keeps all earlier ones; a macro already in the mark is not eligible. □

Lemma 11 (one group, one step)**.** *Each closing parenthesis token is consumed by at most one invocation. Hence, while scanning a stream V, the number of trick steps whose closing parenthesis comes from V is at most the number of `)` in V, so at most $|V|$.*

Proof. The function rule removes the matched parenthesis together with the invocation. □

Definition 12. A *pass* of level N is a maximal run of trick steps of macros of one pass set of level N that consume groups of one outer stream V. By Lemma 11 a pass has at most $|V|$ steps; a re-invocation of the same pass set inside the pass ("self call") consumes groups of the same V and thus only continues it, never multiplies it.

Theorem 13 (upper bound)**.** *Let $\Lambda = \max \mathrm{lev}$ over $\mathcal{M}$. There are constants c, d, p depending on Γ such that, for input length n, both the number of rewrite steps and the output length are at most $\Phi_\Lambda^{(p)}(c\,n + d)$, where*

$$\Phi_0(x) = 2x, \qquad \Phi_N(x) = \Phi_{N-1}^{(x)}(x) \quad (N \ge 1).$$

In particular every function computed by preprocessing a fixed Γ is primitive recursive; Φ_N lies at row $N+2$ of the Ackermann hierarchy (row 3: exponentiation, row 4: tetration, …): $\Phi_1(x) = x\,2^x$; the bound, a fixed iterate of it, lies in the Grzegorczyk class $\mathcal{E}^{\Lambda+2}$.

Remark 14. The iterate is needed: `EXP2(s)`=`EXPONENTIAL(EXPONENTIAL(s))` has $\Lambda = 1$ and yields 2^{2^n} groups ($n = 1, 2, 3, 4 \mapsto 4, 16, 256, 65536$, `exp2_check.c`), which no $\Phi_1(cn+d)$ bounds; a fixed number of passes of one level composes to a fixed iterate of that level. Checked in Lean (`lean/Phi.lean`): $\Phi_1(x) = x\,2^x$; $\Phi_1(cn+d) < 2^{2^n}$ for some n, for all c, d; and what the proof below uses — each Φ_N is monotone with $x \le \Phi_N(x)$, and $(\Phi_N^{(p)})^{(x)}(y) \le \Phi_{N+1}(p\,y)$ for $p \ge 1$, $x \le y$.

Proof of Theorem 13. Induction on N for streams in which only macros of level $\le N$ occur as eligible heads; write $B_N = \Phi_N^{(p_N)}$ for the bound at level N. $N = 0$: no pass sets with cycles are reachable, the run is trick-free relative to every expansion, and Proposition 6 gives linear length and constant steps per invocation; at most n heads are present, so both quantities are $\le c\,n + d \le \Phi_0(cn + d)$ after adjusting constants ($p_0 = 1$). $N \ge 1$: scan a stream V, $|V| = x$. Steps of level $< N$ are handled by the induction hypothesis on the stream they currently operate on. A level-N pass makes at most x steps (Lemma 11); each step emits a bounded body whose names are of level $< N$ (calls into the own pass set continue the pass, by Lemma 10 and the definition) and may wrap the current material as an argument of such a macro, so the length after i steps is bounded by the i-fold application of the level-$(N-1)$ bound B_{N-1}. After at most x steps the length is $\le B_{N-1}^{(x)}(y) \le \Phi_N(p_{N-1}\, y)$ with $y = cx + d \ge x$; the number of steps is bounded by the same expression, since every step needs at least one token of the current stream. A bounded number (depending on Γ) of passes and of level-N invocations in sequence — `EXP2` above — composes to a fixed iterate: $B_N = \Phi_N^{(p_N)}$ on a linear argument, by monotonicity. □

4 Lower bound (constructions of `ppinf.h`)

Sequences are encoded as $(a_1)(a_2)\dots(a_n)$. All constructions below are taken from `ppinf.h` (`github.com/j4niwzis/ppinf`, commit `3e15f7d`); outputs were checked with `clang -E`.

- *Pass with sentinel.* `X_A(…) … X_B` and `X_B(…) … X_A` alternate; every group of the sequence is consumed by one trick step; the last emitted name is pasted with `_END` by `CAT` into an empty object macro `X_A_END`/`X_B_END`. Instances: `WRAP_SEQ`, `REMOVE_EDGES`, `SLIDE_3_A/B`, `REVERSE_A/B`, `NESTED_*`, `GEN_U_A/B`, `GEN_MULTIPLY_2_A/B`.
- *Deferral.* `DELAY_X_k()` produces `X` only after k further rescans (`DELAY_OPEN_BRACE_k`, `DELAY_CLOSE_BRACE_k`, `DELAY_COMMA_k`, `DELAY_OUT_k`); this places parentheses so that heads produced in one rescan meet groups of outer origin in the next.
- *Branching.* `IF_ELSE(c,t,f)` selects by whether `c` expands to a probe `0,` (e.g. `CAT(TEST_NULL_, x)` with `TEST_NULL_NULL` $\mapsto$ `0,`); likewise `TEST_RIGHT_EDGE_*`, `TEST_BRACE`.
- *Sequence operations.* `SLIDE_2`, `SLIDE_3`, `ZIP`, `REVERSE`, `EQUAL_LENGTH`; e.g. `ZIP((a)(b)(c),(d)(e)(f))` = `(a,d)(b,e)(c,f)`, `EQUAL_LENGTH((a)(b)(c),(c)(d))` = `NO`.

Example 15 (level 1: exponentiation)**.** `MULTIPLY_2(s)` = `s s` is level 0. The pass `GEN_MULTIPLY_2_A` emits `MULTIPLY_2(` once per group, the pass `GEN_BRACE_2_A` emits the matching `)`, and

```
EXPONENTIAL(s) = CAT(GEN_MULTIPLY(s)(~) GEN_BRACE_2_A s, _END)
```

yields 2^n groups `(~)` for $|s| = n$ (checked: $n = 1, 2, 3, 5 \mapsto 2, 4, 8, 32$).

Example 16 (level 2: tetration)**.** With fresh paste macros `TCAT` (a level needs its own helpers, otherwise they are hidden while the lower level expands) and the passes `TGEN_EXP_A/B` (emitting `EXPONENTIAL DELAY_OPEN_BRACE_1()`) and `TCLOSE_A/B` (emitting `)`):

```
TOWER(s) = TCAT(TCAT(TGEN_EXP_A s, _END)(~) TCLOSE_A s, _END)
```

yields $2 \uparrow\uparrow n$ groups (checked: $n = 1, 2, 3, 4 \mapsto 2, 4, 16, 65536$).

Theorem 17 (lower bound)**.** *For every N there is Γ_N with $\Lambda = N$ whose output length on a sequence of length n is at least row $N + 2$ of the Ackermann hierarchy at n. Hence the output-length functions of preprocessing are cofinal in the primitive recursive functions, and Theorem 13 is tight up to the constants c, d, p.*

Proof. Iterate the step from `EXPONENTIAL` to `TOWER`: given Γ_{N-1} computing g at level $N - 1$, add fresh copies of the pass and paste helpers and a pass emitting $\langle g \rangle$ `DELAY_OPEN_BRACE_1()` once per group together with a closing pass; the result is $g^{(n)}$ applied to a constant sequence, one row higher. Freshness of helpers is required by Lemma 3: helpers shared with an enclosing level carry its mark and are not eligible. □

5 Preprocessing computes exactly the primitive recursive functions

Encode $n \in \mathbb{N}$ as the sequence $\overline{n} = \underbrace{(\tilde{})\cdots(\tilde{})}_{n}$; a tuple is a list of such arguments. A function $f : \mathbb{N}^k \to \mathbb{N}$ is *pp-computable* if there is an environment Γ and a macro F with $E\big(F(\overline{x_1}, \dots, \overline{x_k})\big) = \overline{f(\vec{x})}$ for all $\vec{x}$.

Theorem 18 ($\subseteq$)**.** *Every pp-computable function is primitive recursive.*

Proof. Fix Γ and F. A configuration (the token stream with hide sets) is encoded as a natural number; the one-step relation $\to$ is deterministic and, on codes, primitive recursive (it is a bounded search for the leftmost eligible head and a bounded rewrite). By Theorem 13 the computation on input of length n halts within $T(n) = \Phi_\Lambda^{(p)}(cn + d)$ steps, and every intermediate configuration has size $\le T(n)$; both bounds are primitive recursive. Hence $f(\vec{x}) = \mathrm{decode}\big(\mathrm{step}^{(T(n))}(\mathrm{encode}(F(\overline{\vec{x}})))\big)$ is obtained from primitive recursive ingredients by bounded iteration (iterate step exactly $T(n)$ times), so f is primitive recursive. □

Theorem 19 ($\supseteq$)**.** *Every primitive recursive function is pp-computable. Moreover a definition of recursion depth r is realised with $\Lambda = r$.*

Proof. By structural induction on a primitive recursive term, giving a translation $[\![\cdot]\!]$ into macros.

Base. $[\![\mathbf{0}]\!](\overline{\vec{x}}) =$ (empty); $[\![\mathbf{S}]\!](s) = s\,(\tilde{\ })$; $[\![\pi_i^k]\!]$ selects the i-th argument (one function-like macro). All are level 0 and verified.

Composition $h = f \circ (g_1, \ldots, g_m)$: $[\![h]\!](\vec{s}) = [\![f]\!]\big([\![g_1]\!](\vec{s}), \ldots, [\![g_m]\!](\vec{s})\big)$, the g_j pre-expanded as arguments ($E(A_i)$ in the rule). Each occurrence uses fresh copies of the helper macros of its subterms; by Lemma 3 shared helpers would carry an enclosing mark and be ineligible, so freshness is necessary and sufficient. Level = max over subterms.

Primitive recursion $f(0, \vec{y}) = g(\vec{y})$, $f(n{+}1, \vec{y}) = h(n, f(n, \vec{y}), \vec{y})$: realise the iterator

$$\mathrm{ITER}(\Theta)\big(\overline{n}\big)(\sigma) = \Theta^{(n)}(\sigma),$$

as in `EXPONENTIAL`: two passes, `GEN_Θ_A/B` emitting "Θ `DELAY_OPEN_BRACE_1()`" once per group of $\overline{n}$, and a closing pass (`GEN_BRACE_2_A/B`) emitting the matching `)`; the net term is $\Theta(\Theta(\cdots\Theta(\sigma)))$ with n applications. Take σ to be the state $(\,\overline{\text{counter}}\,,\ \overline{\text{value}}\,,\ \vec{y}\,)$ as a single parenthesised tuple (the inner commas are protected), with counter $= \overline{0}$ and value $= [\![g]\!](\vec{y})$ initially, and

$$\Theta(c, v, \vec{y}) = \big(\,\mathbf{S}(c),\ [\![h]\!](c, v, \vec{y}),\ \vec{y}\,\big).$$

Tuple projection and reassembly use `FIRST_VALUE`/`SECOND_VALUE` of `ppinf.h`. The final value field is read off by a projection. After n iterations the value field is $f(n, \vec{y})$ by induction on n. The pass adds one iteration level on top of $[\![h]\!]$, so Λ increases by one exactly at a recursion, matching the depth bound and Theorem 13.

Verified instances. **ADD** $=$ ITER($\mathbf{S}$) threads the accumulator through $\mathbf{S}$: $\mathtt{ADD}(\overline{a}, \overline{b}) = \overline{a+b}$ (checked $3{+}2, 1{+}4, 0{+}3, 2{+}0 \mapsto 5, 5, 3, 2$). At higher levels `EXPONENTIAL` $=$ ITER($\lambda s.\, s\, s$) gives 2^n (level 1) and `TOWER` $=$ ITER(`EXPONENTIAL`) gives $2 \uparrow\uparrow n$ (level 2). □

Corollary 20. *The functions computed by the C preprocessor are* exactly *the primitive recursive functions; a function of recursion depth r is realised at level r, so the growth attainable at level N is precisely row $N{+}2$ of the Ackermann hierarchy. No single Γ computes a function that dominates all primitive recursive functions, because Λ is fixed by the source and expansion creates no definitions: diagonalisation across levels within one run is impossible.*

Remark 21. Theorems 13 and 17 give: preprocessing computes only primitive recursive functions, and its growth reaches every row of the Ackermann hierarchy, but no single Γ reaches all rows, since levels are fixed by the source. Diagonalisation over levels within one run is impossible.